

Scalable telomere-to-telomere assembly for diploid and polyploid genomes with double graph

Paper Information

- **Paper ID:** 2306_03399v1
 - **Data Type:** PacBio HiFi长读长测序、ONT超长读长测序、Hi-C测序、短读长测序（三人组数据）
 - **Organism:** 人类(Homo sapiens)、拟南芥(Arabidopsis thaliana)、马铃薯(Solanum tuberosum, autotetraploid)
 - **Pages Analyzed:** 14
 - **Figures Extracted:** 0
-

Analysis Pipeline

Step 1: 超长读长序列过滤

Description: 过滤掉过短的ONT超长读长序列，以减少组装错误并缩短运行时间。根据样本类型设置不同长度阈值。

Tools: seqkit

Input: 原始ONT ultra-long reads (FASTA格式)

Output: 过滤后的超长读长序列 (FASTA格式)

Parameters: - min_length_year2 : 1000000 (100kb) -
min_length_year1_plant : 500000 (50kb) - seqkit_version : 2.3.0

Example Command:

```
# HPRC Year 2样本
seqkit seq -m1000000 <ultra-long-reads.fasta>

# HPRC Year 1和植物样本
seqkit seq -m500000 <ultra-long-reads.fasta>
```

Best Practice Note: 对于人类Year 2数据集，使用 $\geq 100\text{kb}$ 的读长；对于Year 1和植物数据集，使用 $\geq 50\text{kb}$ 的读长。这确保了足够的读长覆盖度，同时避免过短读长导致的组装错误。

Step 2: 三人组分箱索引构建

Description: 使用父母本短读长数据构建父本和母本的k-mer数据库，用于后续的三人组分箱组装。

Tools: yak

Input: 父本和母本的短读长测序数据 (FASTQ格式)

Output: 父本和母本的k-mer数据库 (.yak文件)

Parameters: - yak_version : 0.1-r62-dirty - kmer_size : 37 (默认) - threads :

Example Command:

```
# 构建父本索引
yak count -b37 -t <nThreads> -o <pat.yak> <paternal-short-reads.fastq>

# 构建母本索引
yak count -b37 -t <nThreads> -o <mat.yak> <maternal-short-reads.fastq>
```

Best Practice Note: 使用37-mer进行计数是标准做法，可以平衡特异性和灵敏度。建议使用多线程加速处理。

Step 3: Hi-C分相组装 (hifiasm UL)

Description: 使用HiFi读长、超长读长和Hi-C读长进行单样本的端粒到端粒单倍型解析组装。

Tools: hifiasm

Input: HiFi读长(FASTA)、过滤后的超长读长(FASTA)、Hi-C读长(R1/R2 FASTA)

Output: 单倍型解析的组装结果 (包括.gfa图文件和.fasta序列文件)

Parameters: - `hifiasm_version`: 0.19.4-r587 - `homozygous_coverage`: (需根据数据估算) - `threads`: - `preset`: 默认参数

Example Command:

```
hifiasm -o <outputPrefix> -t <nThreads> \
--h1 <HiC-reads-R1.fasta> --h2 <HiC-reads-R2.fasta> \
--hom-cov <homozygous coverage> \
--ul <ultra-long-reads.fasta> <HiFi-reads.fasta>
```

Best Practice Note: 需要准确估计纯合覆盖度(--hom-cov)参数，这对组装质量至关重要。Hi-C读长用于单倍型分相，产生两个单倍型组装。

Step 4: 三人组分箱组装 (hifiasm UL)

Description: 结合父母本数据，使用三人组分箱策略进行单倍型解析组装。

Tools: hifiasm

Input: HiFi读长、超长读长、父本/母本yak索引文件

Output: 父本和母本单倍型的组装结果

Parameters: - `hifiasm_version`: 0.19.4-r587 - `parental_databases`: 和 - `homozygous_coverage`: - `threads`:

Example Command:

```
hifiasm -o <outputPrefix> -t <nThreads> \  
-1 <pat.yak> -2 <mat.yak> \  
--ul <ultra-long-reads.fasta> \  
--hom-cov <homozygous coverage> <HiFi-reads.fasta>
```

Best Practice Note: 三人组组装通常比Hi-C分相更准确，特别是在复杂区域。需要父母本短读长数据。

Step 5: 单倍体基因组组装（拟南芥）

Description: 对于纯合度高的样本（如拟南芥Col-0），使用简化模式进行单倍体组装。

Tools: hifiasm

Input: HiFi读长、超长读长

Output: 单倍体组装结果

Parameters: - hifiasm_version:0.19.4-r587 - purge_level:-l0（关闭减量） - threads:

Example Command:

```
hifiasm -o <outputPrefix> -t <nThreads> -l0 \  
--ul <ultra-long-reads.fasta> <HiFi-reads.fasta>
```

Best Practice Note: -l0参数适用于近交系或高度纯合的样本，避免过度清除导致的有意义序列丢失。

Step 6: 多倍体基因组组装（马铃薯）

Description: 使用遗传图谱信息对同源多倍体基因组进行单倍型解析组装。

Tools: hifiasm

Input: HiFi读长、超长读长、遗传图谱文件

Output: 四个单倍型的组装结果

Parameters: - `hifiasm_version` : 0.19.4-r587 -
`homozygous_coverage` : 116 (马铃薯特定参数) -
`duplication_level` : -D10 - `genetic_map` : -5 - `threads` :

Example Command:

```
hifiasm -o <outputPrefix> --hom-cov 116 -D10 -t <nThreads> \  
-5 <genetic-map> \  
--ul <ultra-long-reads.fasta> <HiFi-reads.fasta>
```

Best Practice Note: 多倍体组装需要遗传图谱信息(-5)和特殊的重复水平参数(-D)。同源覆盖度需要根据具体物种调整。

Step 7: Verkko组装 (对比方法)

Description: 使用Verkko进行端粒到端粒组装作为对比基准。

Tools: Verkko, gfase, Meryl

Input: HiFi读长、超长读长、（可选）父母本短读长或Hi-C读长

Output: 未分相或分相后的组装结果

Parameters: - `verkko_version` : 1.3.1 - `gfase_version` : 未明确版本 -
`meryl_version` : 未明确版本

Example Command:

单倍体组装

```
verkko -d <outDir> --hifi <HiFi-reads.fasta> --nano <ultra-long-reads.fasta>
```

三人组组装

```
verkko -d <outDir> --hifi <HiFi-reads.fasta> --nano <ultra-long-reads.fasta> \  
--hap-kmers <mat_hapmer_db> <pat_hapmer_db> trio
```

Best Practice Note: Verkko使用不同的组装策略（multiplex de Bruijn graph），需要不同的参数设置。对于Hi-C分相，需要结合gfase工具链。

Step 8: 基因完整性评估 (asmgene)

Description: 使用cDNA参考评估人类基因组组装中的基因完整性和错误。

Tools: minimap2, paftools.js

Input: 参考基因组(CHM13v2)、cDNA序列、组装的contig序列

Output: 基因完整性统计 (完整、缺失、重复、片段化基因数量)

Parameters: - minimap2_version: 2.24-r1122 - alignment_preset: splice:hq (适用于cDNA比对) - identity_threshold: 0.97 (-i.97)

Example Command:

```
# 比对参考基因组
minimap2 -cxsplice:hq -t <nThreads> <ref.fa> <cDNAs.fa> > <ref.paf>

# 比对组装结果
minimap2 -cxsplice:hq -t <nThreads> <asm_contig.fa> <cDNAs.fa> > <asm.paf>

# 评估基因完整性
paftools.js asmgene -a -i.97 <ref.paf> <asm.paf>
```

Best Practice Note: 使用97%的比对 identity 阈值来定义基因的存在与否。需要首先对参考基因组进行比对作为基准。

Step 9: 基因组完整性评估 (BUSCO)

Description: 使用单拷贝直系同源基因评估非人类基因组组装的完整性。

Tools: BUSCO

Input: 组装的基因组序列(FASTA格式)

Output: BUSCO评分 (完整、缺失基因比例)

Parameters: - busco_version: 5.4.4 - mode: genome - lineage_dataset: brassicales_odb10 (拟南芥) 或 solanales_odb10 (马铃薯) - threads:

Example Command:

```
busco -i <asm.fa> -m genome -o <outDir> -c <nThreads> -l <lineage_dataset>
```

Best Practice Note: 选择合适的谱系数据库对评估结果至关重要。
BUSCO评估过滤掉<500kb的contig以聚焦于染色体级别的组装。

Step 10: 分相准确性评估

Description: 评估组装结果的单倍型分相准确性。

Tools: yak

Input: 父母本k-mer数据库、组装后的contig序列

Output: 转换错误率和Hamming错误率

Parameters: - `yak_version`: 0.1-r62-dirty - `kmer_size`: 31 (用于分相评估) - `threads`:

Example Command:

```
# 人类基因组评估
yak trioeval -t <nThreads> <paternal.yak> <maternal.yak> <asm_contig.fa>

# 马铃薯基因组评估
使用单倍型特异性HiFi读长作为标记
```

Best Practice Note: 转换错误率衡量相邻标记的错误，Hamming错误率衡量总体标记错误率。对于三人组组装，使用父母本短读长数据；对于多倍体，需要单倍型特异性标记。

Step 11: 端粒到端粒(T2T) contig检测

Description: 检测能够覆盖完整染色体（包含两端端粒）的contig。

Tools: HPRC workflow, minimap2 (隐含)

Input: 组装结果、参考基因组

Output: T2T contig数量统计

Parameters: - `reference_human`: CHM13v2 - `reference_plants`: 同数
据集的已发表基因组 - `contig_length_threshold`: 500kb (植物)

Example Command:

使用HPRC工作流程: <https://github.com/biomonika/HPP/blob/main/assembly/w>

Best Practice Note: T2T contig必须比对到参考染色体的完整长度且两端都检测到端粒序列。该评估对染色体级别组装质量至关重要。

Step 12: 云计算成本优化

Description: 在Google Cloud Platform上使用抢占式实例进行成本优化的大规模组装。

Tools: hifiasm, Verkko, Terra平台, WDL workflow

Input: 所有读长数据、临时文件

Output: 最终组装结果

Parameters: - `instance_type`: 抢占式实例 (preemptible instances) -
`max_runtime`: 24小时 - `cost_reduction`: 8-15倍

Example Command:

```
# hifiasm分步运行 (三人组)
# 第1步: 仅分箱
hifiasm -o <outputPrefix> -t <nThreads> --bin-only -1 <pat.yak> -2 <mat.yak>

# 第2步: 整合超长读长
hifiasm -o <outputPrefix> -t <nThreads> --bin-only -1 <pat.yak> -2 <mat.yak>

# 第3步: 最终组装
hifiasm -o <outputPrefix> -t <nThreads> -1 <pat.yak> -2 <mat.yak> --hom-co
```

Best Practice Note: 将组装过程拆分为多个短任务 (<24小时) 以适配抢占式实例的限制。hifiasm的三步策略显著降低了云计算成本, 比Verkko便宜8-15倍。

Databases Used

- GRCh38 (人类参考基因组)
 - CHM13v2 (完整人类参考基因组)
 - cDNA序列数据库 (人类基因评估)
 - brassicales_odb10 (拟南芥BUSCO谱系数据库)
 - solanales_odb10 (马铃薯BUSCO谱系数据库)
 - HPRC Year-1数据集
 - HPRC Year-2数据集
-

Key Methodological Findings

- hifiasm(UL)采用双图框架 (double graph) 策略, 分别构建HiFi和超长读长的string graph, 然后合并, 比现有方法 (如Verkko) 计算成本低一个数量级
 - 在22个人类样本测试中, hifiasm(UL)在低覆盖度数据上产生更连续的组装, 能生成多个染色体的T2T contig, 而Verkko不能
 - Verkko在Hi-C分相时未能将所有contig分配到特定单倍型, 导致常染色体基因缺失更多, 组装完整性较低
 - 在拟南芥组装中, hifiasm(UL)产生5个 $\geq 500\text{kb}$ 的contig (3个T2T), 而Verkko仅产生1个T2T contig
 - hifiasm(UL)成功组装了同源四倍体马铃薯的四个单倍型, 而Verkko不支持多倍体分相
 - 通过利用超长读长覆盖信息, hifiasm(UL)有效解决了string graph中的contained read问题, 保留了关键的contained读长
 - 整数图 (integer graph) 策略避免了超长读长之间的all-vs-all比对, 显著提高了计算效率
 - 在云计算环境下, hifiasm(UL)比Verkko成本效益高8-15倍, 适合群体规模的T2T组装项目
-

